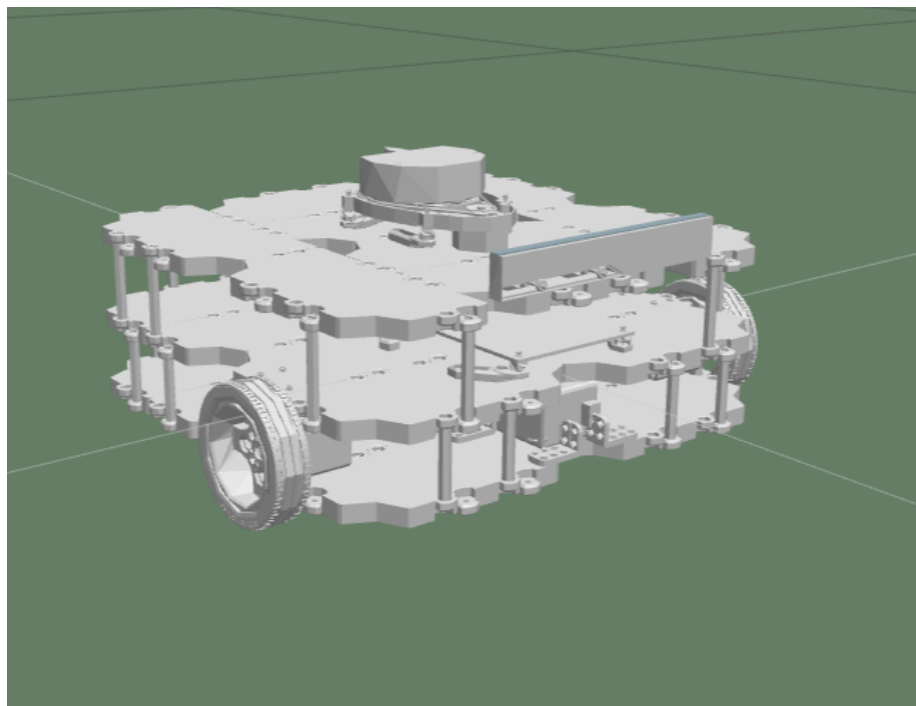


RA: Robòtica Mòbil 2026

Pràctiques

3- Navegació en ROS



Dr. Anaís Garrell Zulueta

Departament d'Enginyeria de Sistemes, Automàtica i Informàtica Industrial

CONTINGUTS

1. Què Aprendre?	2
2. Turtlebot	2
2. Repassem com simular els Turtlebot	2
3. Movem el robot publicant topics	4
3.1 Movem el robot publicant topics a la termina	4
3.2 Movem el robot amb el teclat (repàs)	6
4. Creació de mapes	10
4.1 Guardar el mapa	13
5. Localització del robot	14
6. Path Planning and Obstacle Avoidance	18
7. Afegim obstacles	23
8. Configuració d'un robot	26
9. Stack de Navegació	27
10. Hardware Requirements	28
11. Node move_base	30
12. Resum	30

1. Què Aprendrem?

- Repassarem com simular els turtlebots
- Teleoperem el robot turtlebot (repàs)
- Crearem un mapa i el guardarem
- Localitzarem el robot
- Path Planning and Obstacle Avoidance
- Afegirem obstacles al robot
- Configuració d'un robot
- Stack de Navegació
- Hardware Requirements
- Node move_base

2. Turtlebot

El TurtleBot3 és un robot mòbil de nova generació que és modular, compacte i personalitzable. L'objectiu de TurtleBot3 és reduir dràsticament la mida i baixar el preu de la plataforma sense sacrificar la capacitat, la funcionalitat i la qualitat. Hi ha peces opcionals com el xassís, els ordinadors i els sensors disponibles, i TurtleBot3 es pot personalitzar de diverses maneres. TurtleBot3 té la voluntat d'estar al centre del moviment maker aplicant els últims avenços tècnics de la SBC (Single Board Computer), el sensor de profunditat i la tecnologia d'impressió 3D. TurtleBot3 és un projecte de col·laboració entre Open Robotics, ROBOTIS i més socis com Intel, Onshape, OROCA. Open Robotics és responsable del software i les activitats comunitàries, mentre que ROBOTIS és responsable de la fabricació i la distribució mundial. En aquest curs, aprendràs com pots començar a treballar amb el robot Turtlebot3, explorar les seves funcionalitats i com construir algunes aplicacions ROS interessants per a ell.

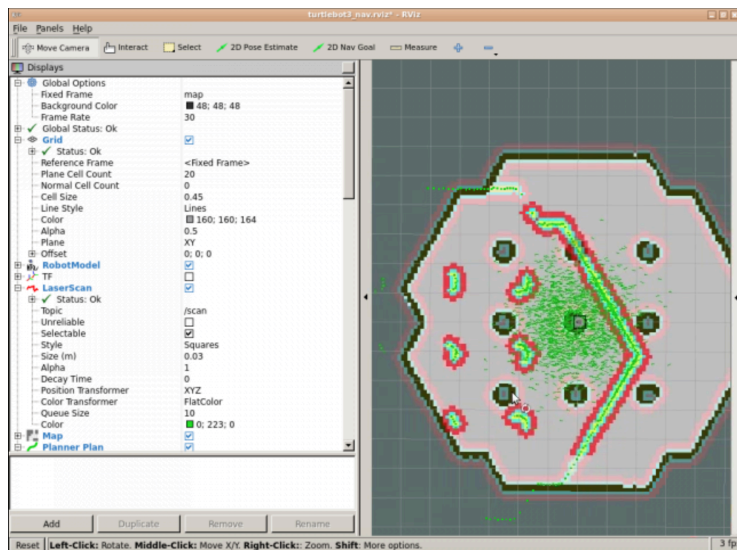
2. Repassem com simular els Turtlebot

- Executeu les següents comandes per a començar el **sistema de navegació**.

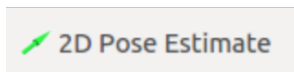
```
T1: $ roscore
T2: $ roslaunch turtlebot3_gazebo turtlebot3_world.launch
T3: $ roslaunch turtlebot3_navigation turtlebot3_navigation.launch
T4: $roslaunch rviz rviz -d `rospack find
turtlebot3_navigation`/rviz/turtlebot3_nav.rviz
```

NOTA: Aquesta última comanda no fa falta, però guardeu-la en el futur per si voleu utilitzar el rviz.

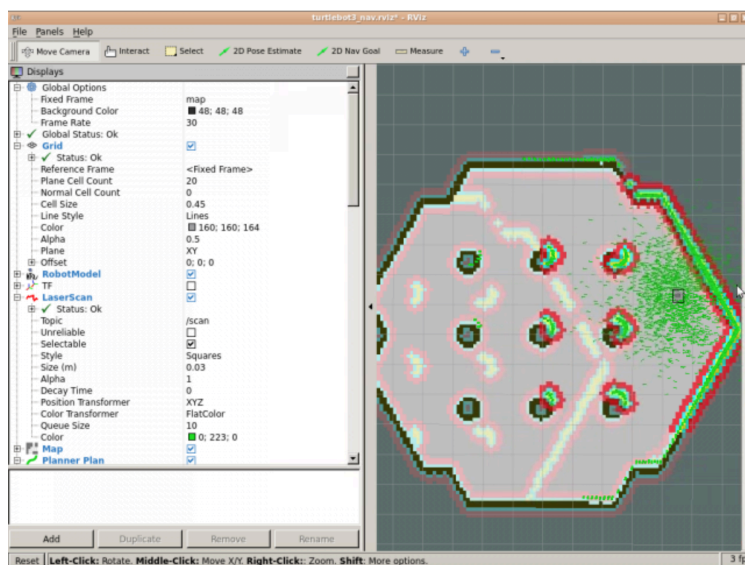
Haurieu de tenir un visualitzador semblant al següent:



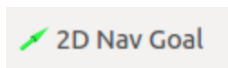
- Utilitzeu l'eina d'estimació de posició 2D a Rviz per localitzar el robot al mapa.



Haurieu d'obtenir una imatge així:

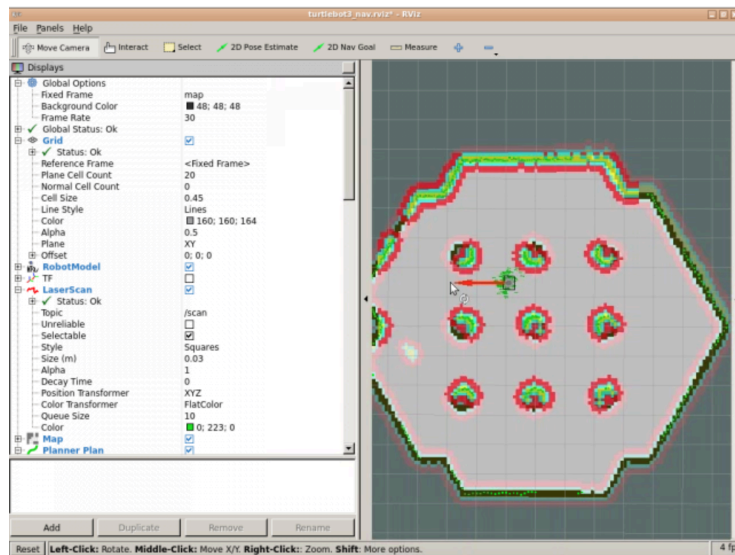


- Utilitzeu l'eina “2D Nav Goal” a Rviz per enviar el robot una posició desitjada (goal).



Ara s'hauria de veure el robot Turtlebot3 dirigint-se a aquesta posició a la simulació. A Rviz, també podeu visualitzar la ruta planificada que segueix.

Veureu que a mesura que es mou el robot es localitza millor.



3. Movem el robot publicant topics

La primera cosa que volem fer quan comencem a treballar amb un robot mòbil és poder moure'l. El robot es pot moure fàcilment utilitzant dos mètodes bàsics:

- Publicant directament als topics
- Utilitzant el teleoperador de teclat

3.1 Movem el robot publicant topics a la terminal

El primer que necessitem saber per moure el robot a través dels topics és quins són els topics que ens permeten fer-ho. Fem una ullada als topics proporcionats per la simulació i veurem si podem entendre-ho.

Nota: Recordeu fer ctrl+C a totes les terminals abans de continuar.

```
T1: $ roscore
T2: $ roslaunch turtlebot3_gazebo turtlebot3_world.launch
T3: $ rostopic list
```

Us apareixen molts topics però ens hem de centrar en els següents:

```

user:~$ rostopic list
/clock
/cmd_vel
/gazebo/link_states
/gazebo/model_states
/gazebo/parameter_descriptions
/gazebo/parameter_updates
/gazebo/set_link_state
/gazebo/set_model_state
/imu
/joint_states
/odom
/rosout
/rosout_agg
/scan
/tf

```

Com sabeu, el topic utilitzat per controlar les velocitats del robot és **/cmd_vel**. Publicant a aquest topic, podrem controlar el moviment del robot Turtlebot3. Anem a veure com funciona:

T4: `$rostopic info /cmd_vel`

```

user:~$ rostopic info /cmd_vel
Type: geometry_msgs/Twist

Publishers: None

Subscribers:
 * /gazebo (http://ip-172-31-13-5:58832/)

```

Com podeu veure, aquest topic utilitza el missatge `geometry_msgs/Twist`. Així que aquest és el tipus de missatge que necessitem enviar al tema per controlar la mà. Doncs, fem una ullada a aquest missatge:

T4: `$rosmmsg show geometry_msgs/Twist`

```

user:~$ rosmmsg show geometry_msgs/Twist
geometry_msgs/Vector3 linear
  float64 x
  float64 y
  float64 z
geometry_msgs/Vector3 angular
  float64 x
  float64 y
  float64 z

```

Tal i com vam treballar en la pràctica anterior, el missatge conté 2 vectors: un per a les velocitats lineals i un altre per a les velocitats angulars. Cada un d'aquests vectors té els seus components corresponents en x, y i z. De tota manera, el Turtlebot utilitza el mecanisme de conducció

diferencial per moure's, és a dir, només cal que ens preocupem pels components lineals x i angulars z, que són els únics que tenen efecte en un robot de conducció diferencial. Així que no tots els camps necessiten ser omplerts, només els que ens interessin.

- Executeu la següent comanda per a generar una estructura del missatge geometry_msgs/Twist.

T4: `$rostopic pub /cmd_vel [TAB][TAB]`

Veureu per pantalla el següent:

```
user:~$ rostopic pub /cmd_vel geometry_msgs/Twist "linear:
  x: 0.0
  y: 0.0
  z: 0.0
angular:
  x: 0.0
  y: 0.0
  z: 0.0"
```

Nota: Si no funciona pot ser que us faltin les cometes tancades “

- Ara, ompliu el missatge amb les velocitats adequades que voleu enviar al robot. Per a aquest exercici, establim la **velocitat lineal en x a 0.5** i la **velocitat angular en z a 0.5** també.
- Finalment, premeu la tecla Enter per enviar el missatge! Si tot ha anat bé, s'hauria de veure el robot Turtlebot3 fent un moviment circular.
- Per aturar el robot només cal tornar a enviar les velocitats en 0.

3.2 Movem el robot amb el teclat (repàs)

Recordeu, per controlar el robot Turtlebot3 amb el teclat fem:

Nota: Fer ctrl+C a totes les terminals abans de continuar.

T1: `$ roscore`
T2: `$ roslaunch turtlebot3_gazebo turtlebot3_world.launch`
T3: `$ roslaunch turtlebot3_teleop keyboard_teleop.launch`

Ara, podeu controlar el robot utilitzant el teclat. Podeu veure com funcionen les diferents tecles a les instruccions següents:

In []:

Control Your Turtlebot!

Moving around:

```
u i o  
j k l  
m , .
```

q/z : increase/decrease **max** speeds by 10%
w/x : increase/decrease only linear speed by 10%
e/c : increase/decrease only angular speed by 10%
space key, k : force stop
anything **else** : stop smoothly

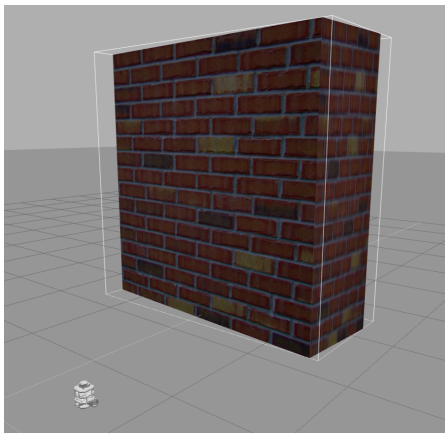
CTRL-C Per sortir

Pel cas del **Turtlebot3 Burger**, que és el que estem utilitzant en aquesta pràctica, porta un LIDAR de 360 graus, que és perfecte per a la navegació i el SLAM

Nota: Recordeu fer ctrl+C a totes les terminals abans de continuar.

```
T1: $ roscore _  
T2: $ roslaunch turtlebot3_gazebo turtlebot3_house.launch
```

- Abans de començar amb l'exercici en sí, el robot ha d'estar prou a prop de la paret que apareix a la simulació:



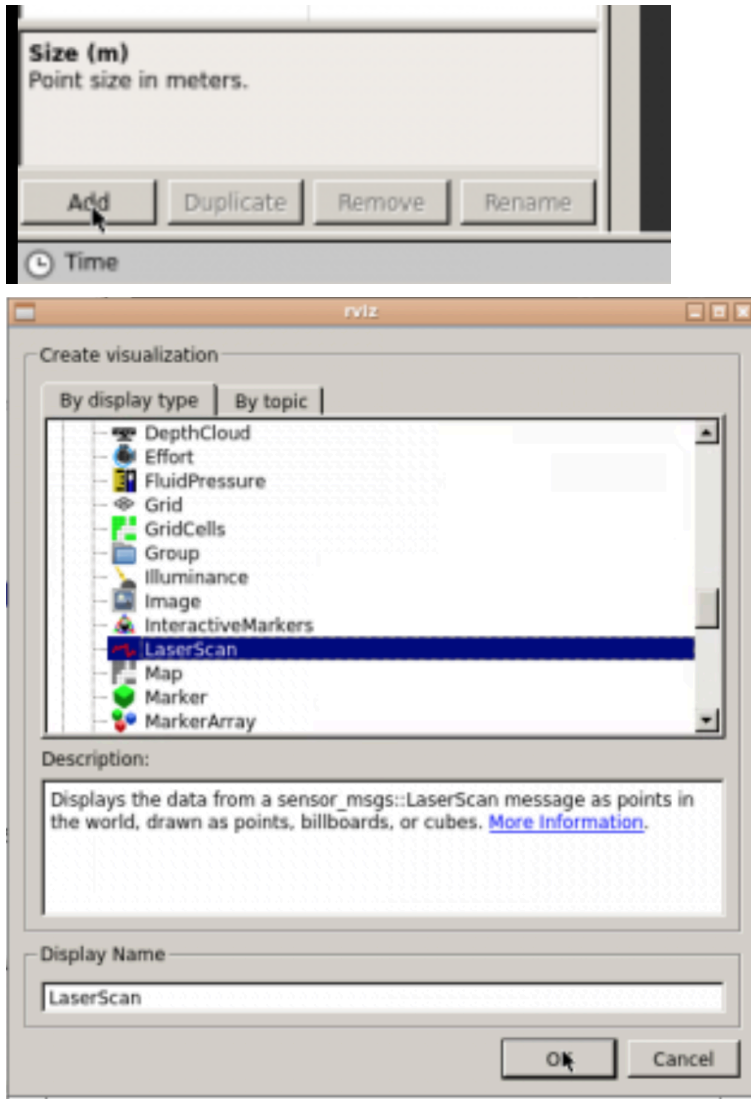
- Llanceu el **rviz**:

```
T3: $ roslaunch turtlebot3_gazebo turtlebot3_gazebo_rviz.launch
```

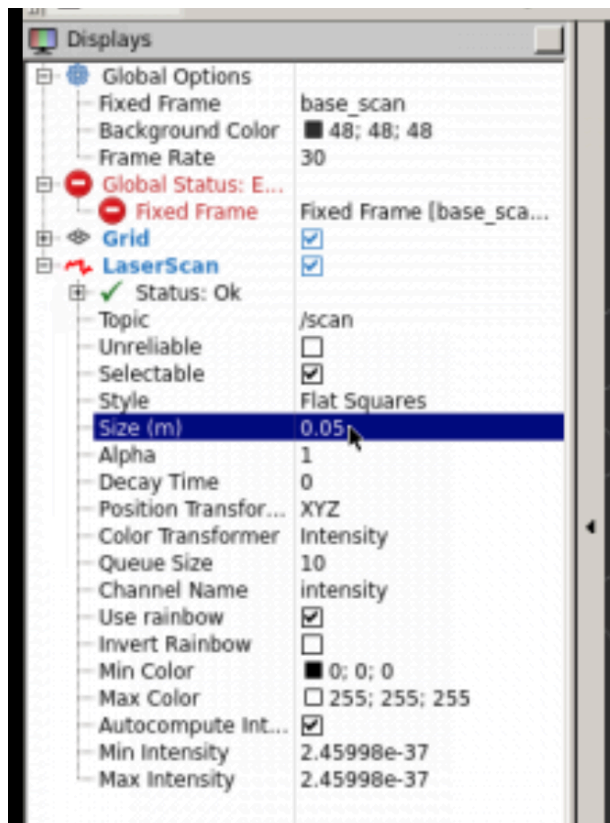
Nota: SI ho voleu fer amb el robot real:

```
T1: $ roscore
T2: $ export TURTLEBOT3_MODEL=burger
T3: $roslaunch rviz rviz -d `rospack find
turtlebot3_navigation`/rviz/turtlebot3_nav.rviz
```

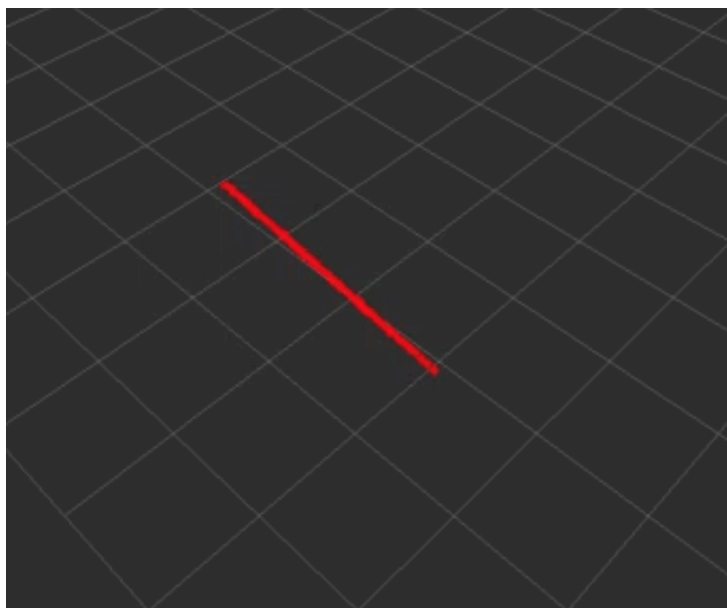
- Cliquez "Add" i seleccioneu LaserScan.



- Configureu la visualització de **LaserScan** amb el topic corresponent.



- Finalment, establiu el marc fix a **base_scan**, que és el topic del làser.



Bàsicament, el que heu fet és **visualitzar les lectures del làser muntat a sobre del robot Turtlebot3**. I ho heu fet utilitzant RViz. A més, podeu visualitzar aquestes dades perquè, de fet, s'estan publicant en un topic.!

Mentre configureu l'element del làser a RViz, heu definit un topic, `/scan`. Això vol dir que aquest és el topic on es publiquen les dades de la lectura del làser.

T4: `$ rostopic echo /scan -n1`

[illegible]

- **angle_min, angle_max i angle_increment:** Aquests valors ens proporcionen informació sobre els angles relacionats amb els raigs del làser. Tenim l'angle on comencen i on acaben (angle_min i angle_max), que ens mostren que els raigs del làser tenen un abast de 360°. A més, el valor angle_increment ens indica quin és l'angle (separació) entre cada raig del làser.
- **range_min i range_max:** Aquests valors proporcionen informació sobre els intervals del làser. Bàsicament, podem veure aquí que el rang d'aquests feixos del làser és de 3,5 metres. Per tant, els objectes més enllà d'aquesta distància no seran detectats pel làser.
- **ranges:** Aquest és el paràmetre més important del missatge. Conté una matriu amb tots els valors associats a les lectures dels feixos del làser.

Així que bàsicament, la part més important del missatge és la matriu de rangs, que en realitat conté les dades de les lectures del sensor del làser. Els altres paràmetres, però, ens ajudaran a interpretar aquests valors. Per exemple, amb els valors dels angles, podem tenir una idea de quina és l'àrea associada a cada una de les lectures del làser contingudes a la matriu de rangs.

4.Creació de mapes.

La primera cosa que necessiteu per realitzar la Navegació ROS és crear un mapa de l'entorn on volgueu navegar. Per això, cal el node [slam_gmapping](#) que proporciona el “stack de Navegació”.

- Creeu un paquet on posarem els arxius relatius a la navegació:

```
T1: $ roscd; cd ../; cd src
```

Aquí creeu un paquet que es digui: **t3_navigation_elvostresnom**

- Dins del paquet, creeu dos directoris nous; **launch** i **param**.

- Creem un arxiu launch per a executar el node **slam_gmapping**:

start_mapping.launch

```
<launch>
  <!-- Turtlebot3 ->

  <arg name="model" default="$(env TURTLEBOT3_MODEL)" doc="model type
[burger, waffle, waffle_pi]"/>
  <include file="$(find turtlebot3_bringup)/launch/turtlebot3_remote.launch" />

  <!-- Gmapping -->
  <node pkg="gmapping" type="slam_gmapping" name="turtlebot3_slam_gmapping"
output="screen">
    <param name="base_frame" value="base_footprint"/>
    <param name="odom_frame" value="odom"/>
    <param name="map_update_interval" value="2.0"/>
    <param name="maxUrange" value="4.0"/>
    <param name="minimumScore" value="100"/>
    <param name="linearUpdate" value="0.2"/>
    <param name="angularUpdate" value="0.2"/>
    <param name="temporalUpdate" value="0.5"/>
    <param name="delta" value="0.05"/>
    <param name="lskip" value="0"/>
    <param name="particles" value="120"/>
    <param name="sigma" value="0.05"/>
    <param name="kernelSize" value="1"/>
    <param name="lstep" value="0.05"/>
    <param name="astep" value="0.05"/>
    <param name="iterations" value="5"/>
    <param name="lsigma" value="0.075"/>
    <param name="ogain" value="3.0"/>
    <param name="srr" value="0.01"/>
    <param name="srt" value="0.02"/>
    <param name="str" value="0.01"/>
    <param name="stt" value="0.02"/>
    <param name="resampleThreshold" value="0.5"/>
    <param name="xmin" value="-10.0"/>
    <param name="ymin" value="-10.0"/>
    <param name="xmax" value="10.0"/>
    <param name="ymax" value="10.0"/>
    <param name="llsamplerange" value="0.01"/>
    <param name="llsamplestep" value="0.01"/>
    <param name="lasamplerange" value="0.005"/>
```



```
<param name="lasamplestep" value="0.005"/>

</node>
</launch>
```

In []:

Els paràmetres més importants aquí són:

- **maxUrange**: Aquest paràmetre estableix fins a quina distància el làser serà considerat per crear el mapa. Un rang més gran crearà mapes més ràpidament i és menys probable que el robot es perdi. La desavantatge és que consumeix més recursos.
- **throttle_scans**: Molt útil per reduir el consum de recursos.

Per a més informació sobre tots aquests paràmetres, si us plau, consulteu la documentació del node `slam_gmapping`: <http://wiki.ros.org/gmapping>

- Executem el launch:

```
T1: $ roscore
T2: $ roslaunch turtlebot3_gazebo turtlebot3_world.launch
```

Nota; Recordeu fer l'export del model del robot abans de llençar el gazebo.

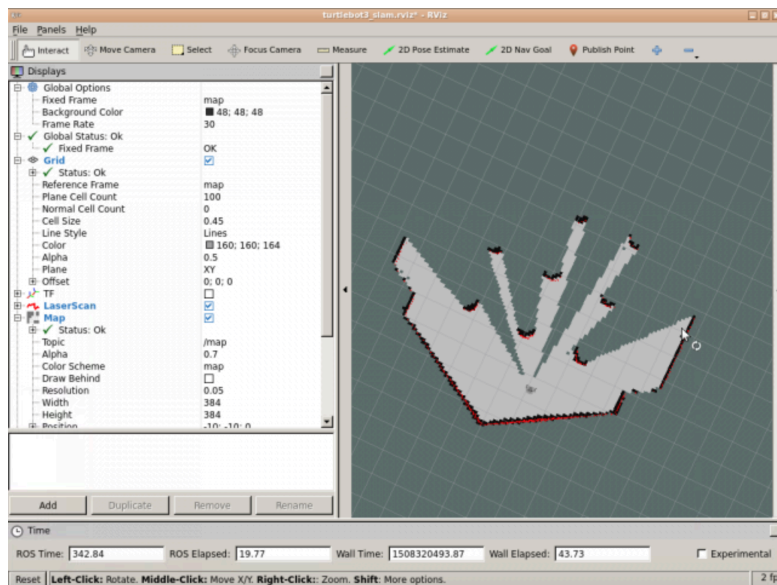
A la T3 podeu

executar el vostre node.

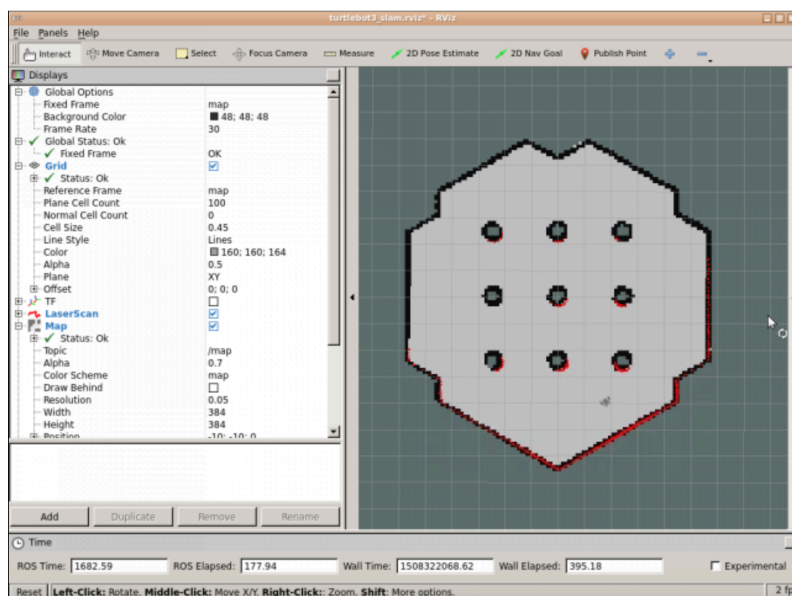
- Ara llancem el RViz per poder visualitzar el procés de mapatge. Per això, tenim dues opcions:
 1. Llançar un RViz bàsic i afegir els elements adequats per visualitzar el procés de mapatge (LaserScan i Mapa).
 2. Podeu utilitzar la següent comanda, que llança una configuració predefinida de RVIZ per al mapatge:

```
T4: $ rosrn rviz rviz -d `rospack find
turtlebot3_slam`/rviz/turtlebot3_slam.rviz
```

NOTA: Recordeu afegir els topic map i scan al rviz.



- Ara podeu començar a moure el robot per l'entorn per generar un mapa complet. Al final, haurieu de tenir alguna cosa semblant a això:



- També podeu jugar amb els valors del paràmetre **maxUrange** per veure com afecta el procés de mapatge.

IMPORTANT: NO TANQUEU res quan acabeu amb l'exercici (heu creat el mapa complet). Haurieu de treballar amb això.

4.1 Guardar el mapa.

Un altre dels paquets disponibles al Stack de Navegació ROS és el paquet **map_server**. Aquest paquet proporciona el node **map_saver**, que ens permet accedir a les dades del mapa des d'un Servei ROS i desar-lo en un fitxer.

Podeu desar el mapa construït en qualsevol moment utilitzant la següent ordre:

```
roslaunch map_server map_saver -f name_of_map
```

NOTA: fixeu-vos el directory on esteu perquè és allà on es guardarà el mapa.

Aquesta ordre obtindrà les dades del mapa del topic `map` i les escriurà en 2 fitxers, `name_of_map.pgm` i `name_of_map.yaml`.

- Guardar el mapa de l'exercici anterior:

```
roscd t3_navigation_elvostrenom;  
mkdir maps;  
cd maps;  
roslaunch map_server map_saver -f my_map;
```

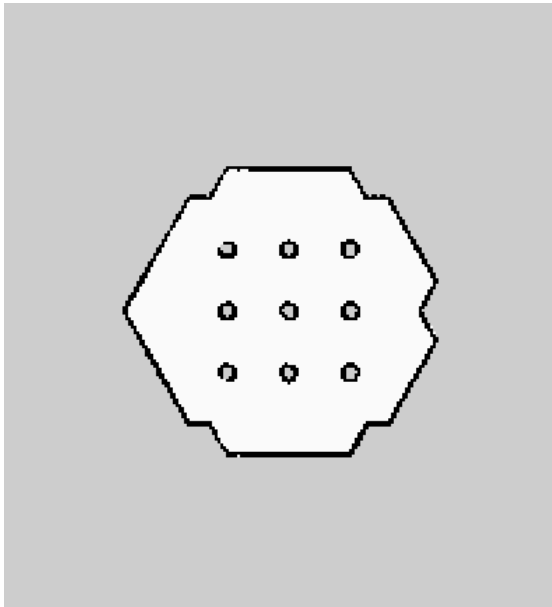
Haurieu de tenir dos arxius nous: **my_map.yaml** and **my_map.pgm**.

El fitxer PGM és el que conté les dades d'ocupació del mapa (les dades realment importants), i el fitxer YAML conté metadades sobre el mapa, com les dimensions del mapa i la resolució, o la ruta al fitxer PGM.

my_map.yaml

image: my_map.pgm resolution: 0.050000 origin: [-10.000000, -10.000000, 0.000000] negate: 0
occupied_thresh: 0.65 free_thresh: 0.196

my_map.pgm



5. Localització del robot,

Així que després de generar el mapa, el següent que necessitem fer és poder localitzar el robot en aquest mapa.

Per això, utilitzarem el node **amcl** del Stack de Navegació. Així doncs, tal com has fet amb el procés de mapatge, creem un fitxer de llançament per iniciar aquest node.

- Al vostre paquet, creeu un nou launch per a començar el node de localització.

start_localization.launch

```
<launch>

  <include file="$(find turtlebot3_bringup)/launch/turtlebot3_remote.launch" />

  <arg name="map_file" default="$(find t3_navigation)/maps/my_map.yaml"/>
  <node name="map_server" pkg="map_server" type="map_server" args="$(arg
map_file)" />

  <arg name="scan_topic" default="scan"/>
  <arg name="initial_pose_x" default="0.0"/>
  <arg name="initial_pose_y" default="0.0"/>
  <arg name="initial_pose_a" default="0.0"/>

  <node pkg="amcl" type="amcl" name="amcl">

    <param name="min_particles" value="500"/>
    <param name="max_particles" value="3000"/>
    <param name="kld_err" value="0.02"/>
    <param name="update_min_d" value="0.20"/>
    <param name="update_min_a" value="0.20"/>
    <param name="resample_interval" value="1"/>
    <param name="transform_tolerance" value="0.5"/>
    <param name="recovery_alpha_slow" value="0.00"/>
    <param name="recovery_alpha_fast" value="0.00"/>
    <param name="initial_pose_x" value="$(arg initial_pose_x)"/>
    <param name="initial_pose_y" value="$(arg initial_pose_y)"/>
    <param name="initial_pose_a" value="$(arg initial_pose_a)"/>
    <param name="gui_publish_rate" value="50.0"/>

    <remap from="scan" to="$(arg scan_topic)"/>
    <param name="laser_max_range" value="3.5"/>
    <param name="laser_max_beams" value="180"/>
    <param name="laser_z_hit" value="0.5"/>
    <param name="laser_z_short" value="0.05"/>
    <param name="laser_z_max" value="0.05"/>
    <param name="laser_z_rand" value="0.5"/>

  </node>

</launch>
```

```

<param name="laser_sigma_hit"      value="0.2"/>
<param name="laser_lambda_short"    value="0.1"/>
<param name="laser_likelihood_max_dist" value="2.0"/>
<param name="laser_model_type"      value="likelihood_field"/>

<param name="odom_model_type"       value="diff"/>
<param name="odom_alpha1"           value="0.1"/>
<param name="odom_alpha2"           value="0.1"/>
<param name="odom_alpha3"           value="0.1"/>
<param name="odom_alpha4"           value="0.1"/>
<param name="odom_frame_id"         value="odom"/>
<param name="base_frame_id"         value="base_footprint"/>

</node>
</launch>

```

Els paràmetres més importants aquí són:

- **min_particles, max_particles:** Aquest paràmetre estableix el nombre de partícules que el filtre utilitzarà per localitzar el robot. Com més, més precisa serà la localització, però més recursos consumirà.
- **laser_max_range:** Màxim rang dels raigs del làser.

Per a més informació sobre els paràmetres: <http://wiki.ros.org/amcl>

- Ara, podem començar el launch:.

```

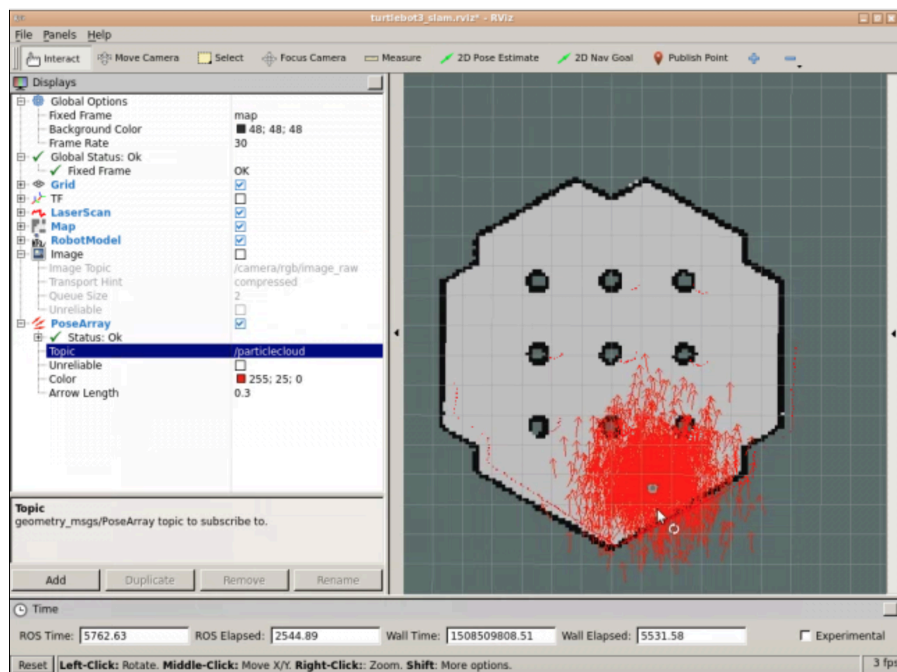
T1: $ roscore
T2: $ roslaunch turtlebot3_gazebo turtlebot3_world.launch

```

Nota; Recordeu fer l'export del model del robot abans de llençar el gazebo.

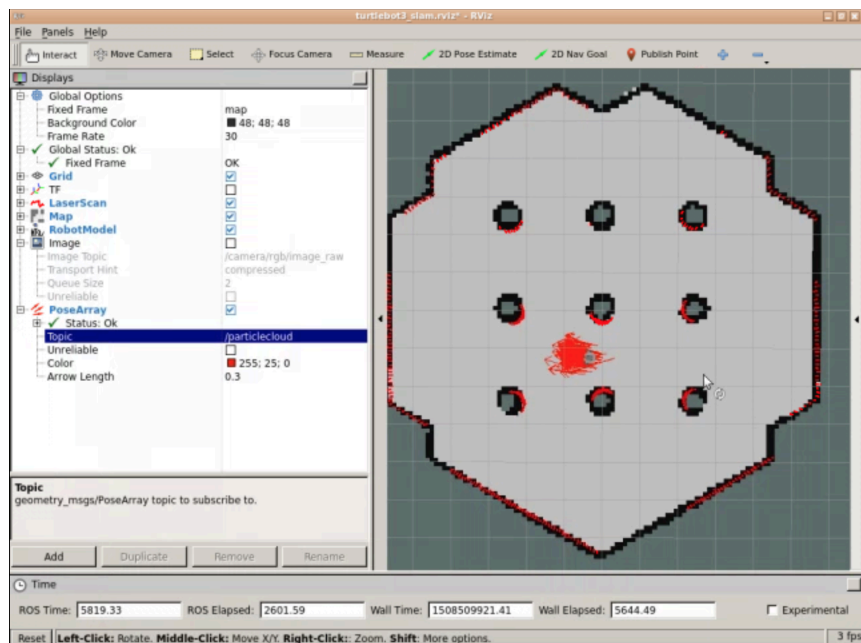
A la terminal 3 executeu el vostre launch.

- Ara llancem RViz per poder visualitzar el procés de localització. Podeu utilitzar la mateixa configuració que va utilitzar per al procés de mapatge, afegint un altre element: **Pose Array** (el topic es diu **particleArray**).



- Ara podeu començar a moure el robot per l'entorn per localitzar-lo. A mesura que mogueu el robot, veureu al RViz com les partícules es van acostant, la qual cosa significa que les posicions estimades del robot s'apropen al lloc real. Això és una prova de com funciona el teu sistema de localització.

Recordeu fer el teleop per desplaçar el robot per l'entorn.



- També pots jugar amb els valors dels paràmetres **min_particles** i **max_particles** per veure com afecta el procés de localització.

6. Path Planning and Obstacle Avoidance

Per fer la planificació de rutes, heu de combinar tot el que heu fet fins ara. A més, haureu d'utilitzar el node **move_base** del Stack de Navegació, que gestionarà tot el sistema de planificació de rutes per vosaltres. Així doncs, com heu fet en els exercicis anteriors, creem el nostre fitxer launch per llançar el sistema de planificació de rutes. Aquesta vegada, però, tindreu una mica més de feina a fer, ja que hi ha molts paràmetres involucrats que haureu de configurar.

Creeu un nou arxiu launch al vostre paquet per a executar el node `move_base`,

start_navigation.launch

```
<launch>
  <arg name="model" default="burger" doc="model type [burger, waffle]"/>

  <!-- AMCL -->
  <include file="$(find t3_navigation)/launch/start_localization.launch"/>

  <!-- move_base -->
  <arg name="cmd_vel_topic" default="/cmd_vel" />
  <arg name="odom_topic" default="odom" />
  <node pkg="move_base" type="move_base" respawn="false" name="move_base"
output="screen">
    <param name="base_local_planner" value="dwa_local_planner/DWAPlanerROS" />

    <rosparam file="$(find t3_navigation)/param/costmap_common_params_$(arg
model).yaml" command="load" ns="global_costmap" />
    <rosparam file="$(find t3_navigation)/param/costmap_common_params_$(arg
model).yaml" command="load" ns="local_costmap" />
    <rosparam file="$(find t3_navigation)/param/local_costmap_params.yaml"
command="load" />
    <rosparam file="$(find t3_navigation)/param/global_costmap_params.yaml"
command="load" />
    <rosparam file="$(find t3_navigation)/param/move_base_params.yaml" command="load"
/>
    <rosparam file="$(find t3_navigation)/param/dwa_local_planner_params.yaml"
command="load" />

    <remap from="cmd_vel" to="$(arg cmd_vel_topic)"/>
    <remap from="odom" to="$(arg odom_topic)"/>
  </node>
</launch>
```

Observem tres parts principals:

- **Map Server:** El node que publica el mapa guardat.
- **AMCL:** El node que localitza el robot basat en les lectures del làser.
- **move_base:** El node que proporcionarà el Path Planning i el Obstacle Avoidance

Per això, només cal crear tots aquests fitxers de paràmetres que s'estan carregant per al node move_base.

- Creació dels arxius de paràmetres per al node move_base:

costmap_common_params_burger.yaml

```
obstacle_range: 2.5
raytrace_range: 3.5
footprint: [[-0.110, -0.090], [-0.110, 0.090], [0.041, 0.090], [0.041, -0.090]]
#robot_radius: 0.105
inflation_radius: 0.15
cost_scaling_factor: 0.5
map_type: costmap
transform_tolerance: 0.2
observation_sources: scan
scan: {data_type: LaserScan, topic: scan, marking: true, clearing: true}
```

Observe que els paràmetres **observation_sources**, que utilitza el topic /scan per llegir les lectures del làser. La càmera no s'utilitza per a la navegació. També observeu que l'abast de traçat de raigs només és de 3,5 metres. Això només serveix per fer les deteccions més ràpides i no tenir en compte àrees que no estan prou a prop.

local_costmap_params.yaml

```
local_costmap:
  global_frame: odom
  robot_base_frame: base_footprint
  update_frequency: 2.0
  publish_frequency: 0.5
  static_map: false
  rolling_window: true
  width: 3.5
  height: 3.5
  resolution: 0.05
  transform_tolerance: 1.0
```


Observeu que el paràmetre **static_map** és False. Això és degut a què el local costmap està construït a partir de lectures de làser, no d'un mapa estàtic.

global_costmap_params.yaml

```
global_costmap:  
  global_frame: map  
  robot_base_frame: base_footprint  
  update_frequency: 2.0  
  publish_frequency: 0.1  
  static_map: true  
  transform_tolerance: 1.0
```

Noteu que el paràmetre **static_map** aquí s'està establint a True. Això és perquè el mapa de cost global es construeix a partir del mapa estàtic que heu creat en els passos anteriors.

move_base_params.yaml

```
shutdown_costmaps: false  
controller_frequency: 3.0  
controller_patience: 1.0  
planner_frequency: 2.0  
planner_patience: 1.0  
oscillation_timeout: 10.0  
oscillation_distance: 0.2  
conservative_reset_dist: 0.10
```

Hi ha un paràmetres generals per al node move_base.

dwa_local_planner_params.yaml

```
DWAPlannerROS:  
  
# Robot Configuration Parameters  
max_vel_x: 0.18  
min_vel_x: -0.18  
  
max_vel_y: 0.0  
min_vel_y: 0.0
```

```
# The velocity when robot is moving in a straight line
```

```
max_trans_vel: 0.18
```

```
min_trans_vel: 0.05
```

```
# trans_stopped_vel: 0.01
```

```
max_rot_vel: 1.8
```

```
min_rot_vel: 0.7
```

```
# rot_stopped_vel: 0.01
```

```
acc_lim_x: 2.0
```

```
acc_lim_theta: 2.0
```

```
acc_lim_y: 0.0
```

```
# Goal Tolerance Parametes
```

```
yaw_goal_tolerance: 0.15
```

```
xy_goal_tolerance: 0.05
```

```
# Forward Simulation Parameters
```

```
sim_time: 3.5
```

```
vx_samples: 20
```

```
vy_samples: 0
```

```
vtheta_samples: 40
```

```
# Trajectory Scoring Parameters
```

```
path_distance_bias: 32.0
```

```
goal_distance_bias: 24.0
```

```
occdist_scale: 0.04
```

```
forward_point_distance: 0.325
```

```
stop_time_buffer: 0.2
```

```
scaling_speed: 0.25
```

```
max_scaling_factor: 0.2
```

```
# Oscillation Prevention Parameters
```

```
oscillation_reset_dist: 0.05
```

```
# Debugging
```

```
publish_traj_pc : true
```

```
publish_cost_grid_pc: true
```

```
# global_frame_id: odom
```

Aquí hi ha alguns paràmetres relatius al local planner.

- Per executar el teu fitxer launch i iniciar el sistema de navegació, utilitza la següent comanda:

T1: `$ roscore`

T2: `$ roslaunch turtlebot3_gazebo turtlebot3_world.launch`

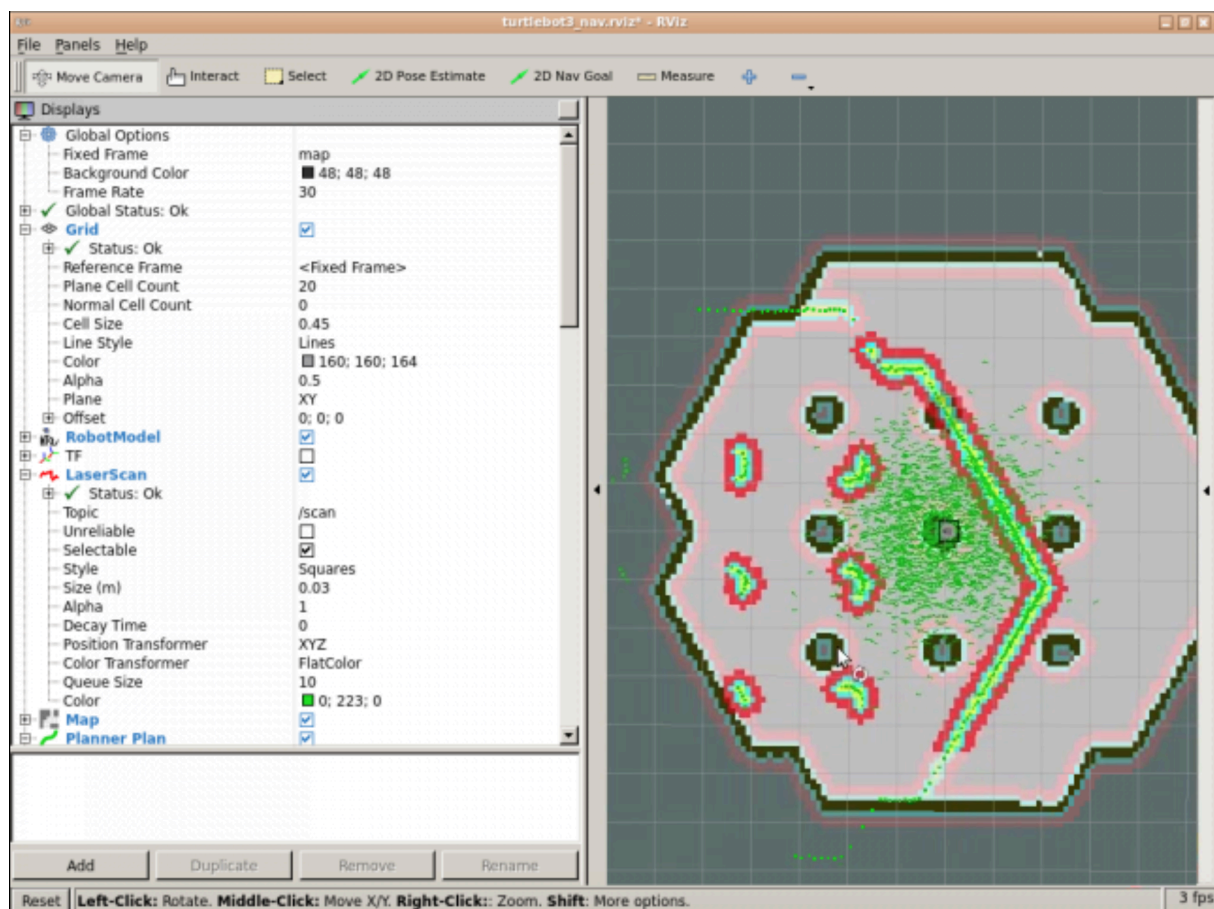
- Això posarà en marxa tots els nodes necessaris per al sistema de navegació amb la configuració que has especificat al teu fitxer launch. Si sorgeixen problemes durant l'execució, verifica els missatges de error i assegura't de corregir qualsevol configuració incorrecta.

T3: `$ roslaunch t3_navigation start_navigation.launch`

- Per executar al path planning: Podeu utilitzar la següent comanda per llançar una configuració predefinida de RVIZ per a la navegació:

T4: `roslaunch rviz rviz -d `rospack find
turtlebot3_navigation`/rviz/turtlebot3_nav.rviz`

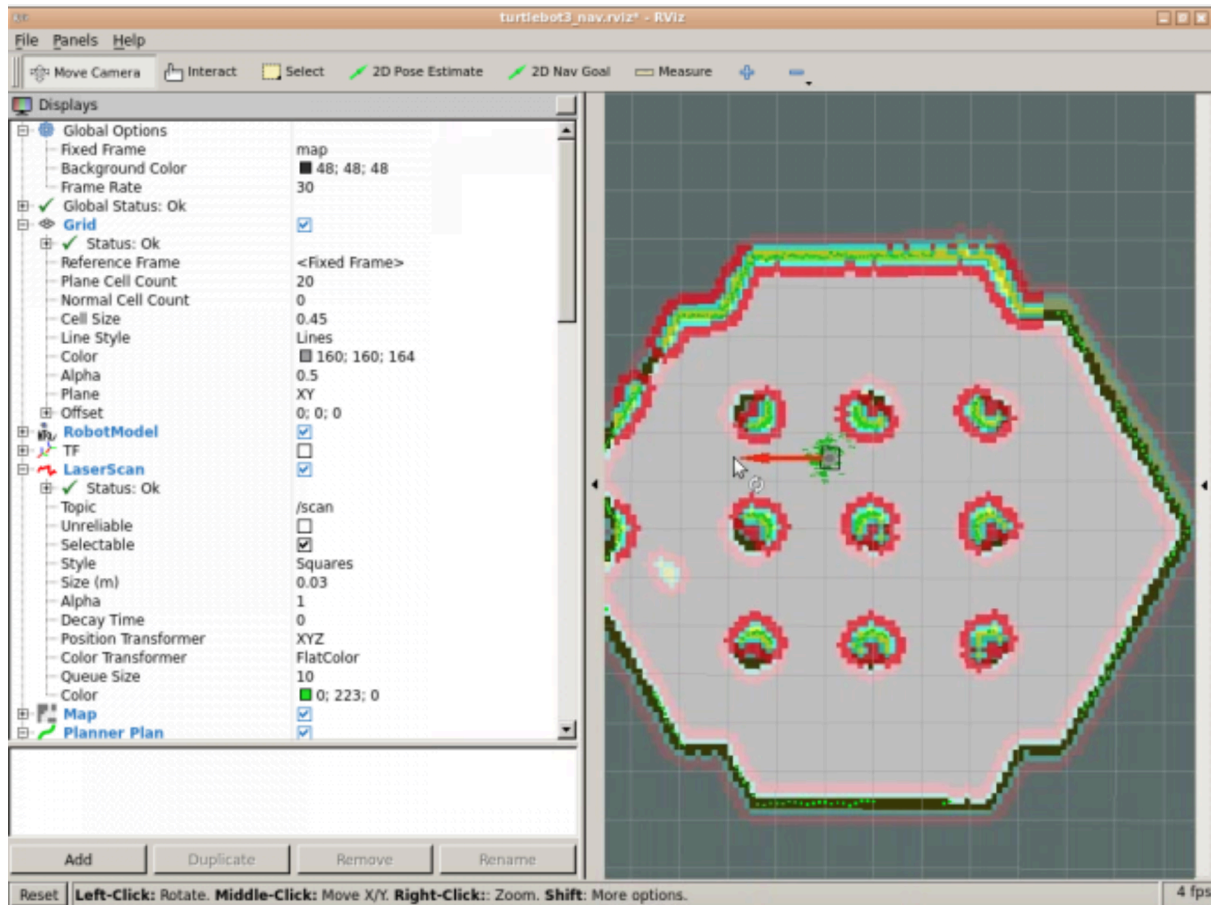
Haurieu de veure una imatge similar a la següent, no us quedarà igual, però jugueu amb les opcions: robot model, laser scan, map, planner plan , global map, local map, costmap, planner, amcl particles, goal...



Per localitzar el robot al mapa, useu l'eina 2D Pose Estimate a RViz. Simplement selecciona la posició aproximada del robot al mapa i arrossega-la fins a la seva posició real.

Per enviar un objectiu (posició desitjada) al robot, utilitzeu l'eina 2D Nav Goal a RViz. Clica a la posició del mapa on vols que el robot vagi i arrossega el cursor fins a la posició desitjada.

Un cop ho hagueu fet, heu de veure el robot Turtlebot3 dirigint-se a aquesta posició a la simulació. A RViz, també podeu visualitzar la ruta planificada que segueix.



També podeu experimentar amb alguns dels valors en els fitxers de paràmetres que heu creat anteriorment per veure com afecten el procés de planificació de rutes. Modificant aquests valors, podreu observar com varia el comportament del sistema de navegació i com afecta a la ruta planificada i al moviment dels robots.

7. Afegim obstacles

Bàsicament, el sistema d'Evitació d'Obstacles descompon la gran imatge (Mapa) en peces més petites, que s'actualitzen en temps real utilitzant les dades que rep dels sensors.

D'aquesta manera, assegura que no es sorprendrà per cap canvi sobtat en l'entorn, o per cap obstacle que aparegui en el camí. Veurem un exemple de com funciona això.

Anem a afegir manualment un objecte a l'entorn:

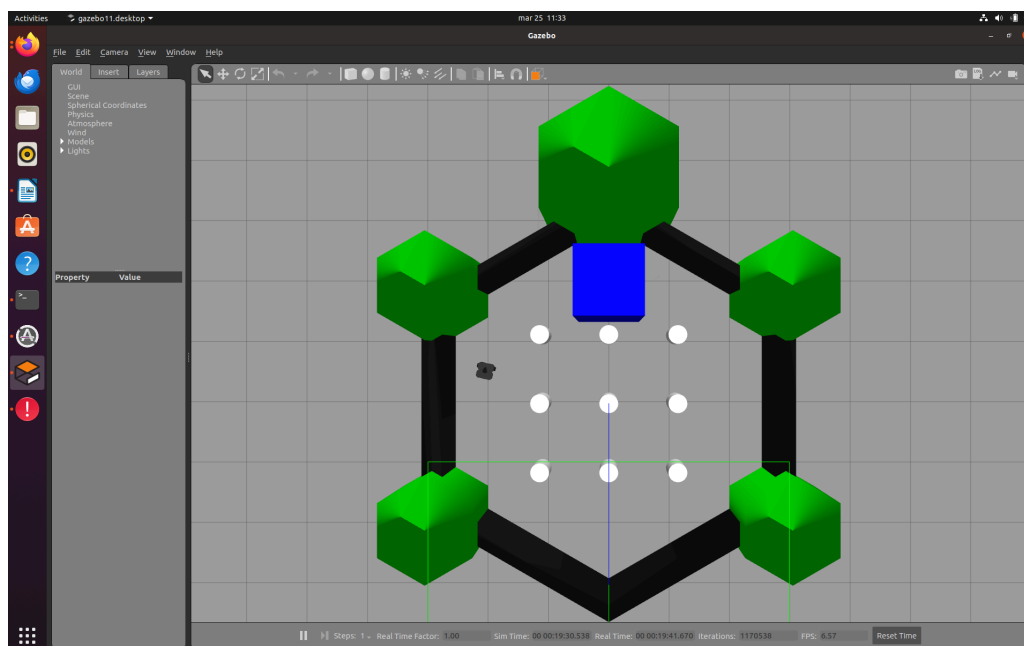
Per copiar el fitxer URDF de l'obstacle al vostre espai de treball, copieu aquest codi, creeu una carpeta urdf a ~/catkin_ws/src/:

Object.urdf

```
<robot name="simple_box">
  <link name="my_box">
    <inertial>
      <origin xyz="2 0 0" />
      <mass value="1.0" />
      <inertia ixx="1.0" ixy="0.0" ixz="0.0" iyy="100.0" iyz="0.0" izz="1.0" />
    </inertial>
    <visual>
      <origin xyz="2 0 1"/>
      <geometry>
        <box size="1 1 2" />
      </geometry>
    </visual>
    <collision>
      <origin xyz="2 0 1"/>
      <geometry>
        <box size="1 1 2" />
      </geometry>
    </collision>
  </link>
  <gazebo reference="my_box">
    <material>Gazebo/Blue</material>
  </gazebo>
</robot>
```

```
roslaunch gazebo_ros spawn_model -file ~/catkin_ws/src/urdf/object.urdf -urdf
-x 0 -y 0 -z 1 -model my_object
```

Si tot funciona bé, hauríeu de veure una caixa blava apareixent a la simulació. Com a exemple:



Per iniciar RViz amb una configuració predefinida, executeu la següent comanda a la terminal:

```
roslaunch turtlebot3_navigation turtlebot3_navigation.launch
```

Utilitzeu les eines 2D Pose Estimate i 2D Nav Goal a RViz (com en l'exemple anterior) per indicar al robot ON és i ON voleu que vagi. Proporcioneu al robot una ubicació que obligui el robot a evitar l'obstacle per comprovar com funciona el sistema.

Establiu un Objectiu de Posició on el robot hagi de evitar de alguna manera l'obstacle que acabeu de generar a la simulació.

NOTA: Al mapa del rviz no surt el dibuix de l'objecte però el làser si el detecta

Nota: ****IMPORTANT** Quan hagueu acabat amb aquest exercici, assegureu-vos de eliminar de nou l'objecte inserit utilitzant la següent ordre:**

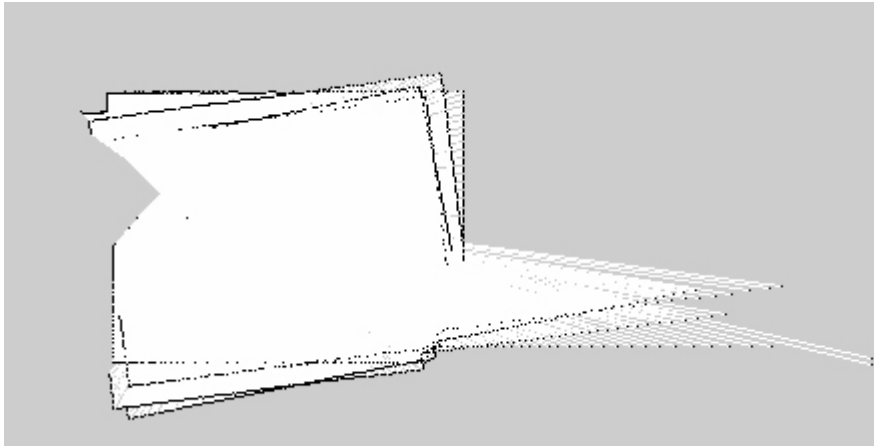
```
rosservice call /gazebo/delete_model "model_name: 'my_object'"
```

Ara mateix, ja heu passat per diferents exemples de cadascuna de les parts involucrades en la Navegació ROS. Però... penseu que tot aquest sistema funcionaria directament, tal com està? Per a qualsevol tipus de robot? La resposta és NO. No ho faria.

Per tenir el sistema de Navegació ROS funcionant correctament, heu de proporcionar alguna dada al sistema, depenent del robot que vulgueu utilitzar i de com estigui construït. És a dir, heu de tenir els vostres robots configurats correctament.

La configuració del robot és extremadament important en tots els mòduls de navegació. Per exemple, en el sistema de mapatge, si no li dius al sistema ON té el robot el làser muntat, quina és l'orientació del làser, quina és la posició de les rodes en el robot, etc., no podrà crear un mapa bo i precís. I com ja podeu saber en aquest punt, si no tenim un bon mapa en la Navegació ROS, NO TENIM RES!

La següent imatge és un exemple de com seria un fitxer de mapa si el làser del robot no està configurat correctament



8. Configuració d'un robot

La configuració i definició del robot es realitza en els fitxers URDF del robot. **URDF** (Unified Robot Description Format) i és un format XML que descriu un model de robot. Defineix les seves diferents parts, dimensions, cinemàtica, dinàmica, sensors, etc. Cada vegada que veieu un robot 3D a ROS, hi ha associat un fitxer URDF.

Per exemple, vegem com està definit el làser del robot Kobuki en el fitxer URDF del robot:

```
<joint name="laser_sensor_joint" type="fixed">
  <origin xyz="0.0 0.0 0.435" rpy="0 0 0"/>
  <parent link="base_link"/>
  <child link="laser_sensor_link"/>
</joint>

<link name="laser_sensor_link">
  <inertial>
    <mass value="1e-5"/>
    <origin xyz="0 0 0" rpy="0 0 0"/>
    <inertia ixx="1e-6" ixy="0" ixz="0" iyy="1e-6" iyz="0" izz="1e-6"/>
  </inertial>
  <collision>
    <origin xyz="0 0 0" rpy="0 0 0"/>
    <geometry>
      <box size="0.1 0.1 0.1"/>
    </geometry>
  </collision>
```

```
<visual>
  <origin xyz="0 0 0" rpy="0 0 0"/>
  <geometry>
    <mesh filename="package://hokuyo/meshes/hokuyo.dae"/>
  </geometry>
</visual>
</link>
```

IMPORTANT: El codi XML mostrat anteriorment és només una secció del fitxer URDF que defineix el robot Kobuki. En aquest cas, és la secció on es defineix el làser.

Com podeu veure, es defineixen diverses coses pel que fa al làser:

- Es defineix la posició i orientació del làser respecte de la base ("base_link") del robot.
- Es defineixen valors relacionats amb la inèrcia.
- Es defineixen valors relacionats amb la col·lisió. Aquests valors proporcionaran la física real del làser.
- Es defineixen valors visuals. Aquests valors proporcionaran els elements visuals del làser. Això és només per a fins de visualització.

Normalment, aquests fitxers es col·loquen dins d'un paquet anomenat **yourrobot_description**.

No obstant això, en aquest curs, no treballarem amb fitxers URDF. Els hem introduït perquè és important conèixer com es gestiona la configuració i definició del robot a ROS, però no anirem més enllà.

Tot i així s'han de configurar molts paràmetres que tenen lloc durant el procés de navegació. Aquests paràmetres et permeten modificar el comportament de les diferents fases involucrades en el procés de navegació (com ara el mapeig, la localització, etc.).

9. Stack de Navegació

Durant els exemples anteriors, heu llançat molts nodes ROS diferents. Cadascun d'aquests nodes ha llançat els seus propis programes de ROS per executar diferents tasques. I tot això s'ha contingut, com sempre passa a ROS, en paquets. Com potser heu suposat, heu estat utilitzant el que es coneix a ROS com el Stack de Navegació.

El Stack de Navegació és un conjunt de nodes i algorismes de ROS que s'utilitzen per moure un robot de manera autònoma d'un punt a un altre, evitant tots els obstacles que el robot pugui trobar pel camí. El Stack de Navegació de ROS ve amb una implementació de diversos algorismes relacionats amb la navegació que poden ajudar-vos a realitzar navegació autònoma en els teus robots mòbils.

El Stack de Navegació prendrà com a entrada la ubicació actual del robot, la ubicació desitjada on el robot vol anar (posició de destí), les dades d'odometria del robot (encoders

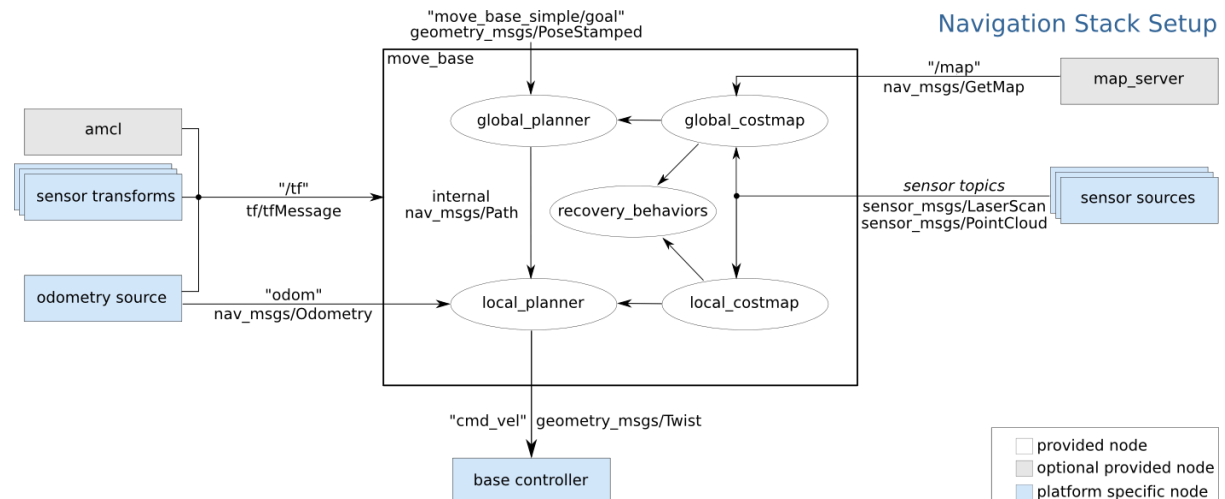
de rodes, IMU, GPS...) i les dades d'un sensor com un làser. A canvi, produirà les comandes de velocitat necessàries i les enviarà a la base mòbil per moure el robot a la posició de destí especificada.

10. Hardware Requirements

El Stack de Navegació de ROS és genèric. Això vol dir que es pot utilitzar amb gairebé qualsevol tipus de robot mòbil, però hi ha algunes consideracions de maquinari que ajudaran que tot el sistema funcioni millor, de manera que s'han de tenir en compte. Aquestes són els requisits:

- El paquet de navegació funcionarà millor en robots de conducció diferencial i holonòmics. A més, el robot mòbil s'hauria de controlar enviant comandes de velocitat en la forma següent: **x, y (velocitat lineal), z (velocitat angular)**
- El robot hauria de tenir un làser planar en algun lloc al voltant del mateix. Aquest és utilitzat per construir el mapa de l'entorn i realitzar la localització.
- El seu rendiment serà millor per a bases mòbils de forma quadrada i circular.

A continuació, hi ha una figura amb els blocs de construcció bàsics del conjunt de navegació extrets del lloc web oficial de ROS (<http://wiki.ros.org/navigation/Tutorials/RobotSetup>). Al final d'aquest curs, sereu totalment capaços de comprendre aquest diagrama i cada bloc que el forma. Per ara, intenta tenir una idea global.



D'acord amb el diagrama mostrat, hem de proporcionar alguns blocs funcionals per poder treballar i comunicar-nos amb el Stack de navegació de ROS. A continuació, són breus explicacions de tots els blocs que cal proporcionar com a entrada al conjunt de navegació de ROS:

- **Odometry source:** Les dades d'odometria d'un robot donen la posició del robot respecte a la seva posició inicial. Les principals fonts d'odometria són els encoders de rodes, les IMU i les càmeres 2D/3D (odometria visual). El valor de l'odometria s'ha de publicar al conjunt de navegació, que té un tipus de missatge de **nav_msgs/Odometry**. El missatge d'odometria pot contenir la posició i la velocitat del robot.
- **Sensor source:** Els sensors s'utilitzen per a dues tasques en la navegació: una per localitzar el robot al mapa (utilitzant, per exemple, el làser) i l'altra per detectar obstacles en el camí del robot (utilitzant el làser, sonars o núvols de punts).
- **sensor transforms/tf:** Les dades capturades pels diferents sensors del robot han de fer referència a un marc de referència comú (normalment el **base_link**) per poder comparar les dades provinents de diferents sensors. El robot hauria de publicar la relació entre el marc de coordenades principal del robot i els diferents marcs dels sensors utilitzant transformacions ROS.
- **base_controller:** La funció principal del controlador de base és convertir la sortida del conjunt de navegació, que és un missatge Twist (**geometry_msgs/Twist**), en velocitats dels motors corresponents per al robot.

Anem ara a recordar com visualitzem la informació dels topics:

```
T1: $ roscore
T2: $ roslaunch turtlebot3_gazebo turtlebot3_house.launch
T3: $ rostopic list
```

Intenteu obtenir tota la informació possible sobre aquests temes, ja que necessitaràs conèixer-los bé.

```
T3: $ rostopic info /cmd_vel
T3: $ rostopic info /odom
T3: $ rostopic info /scan
T3: $ rostopic info /tf
```

També, per obtenir la informació dels missatges:

```
T3: $ rosmmsg show geometry_msgs/Twist
T3: $ rosmmsg show nav_msgs/Odometry
T3: $ rosmmsg show sensor_msgs/LaserScan
T3: $ rosmmsg show tf2_msgs/TFMessage
```

- **/cmd_vel:** Rep la sortida del Conjunt de Navegació i transforma les comandes en velocitats dels motors.
- **/scan:** Proporciona les lectures del Làser al Conjunt de Navegació.
- **/odom:** Proporciona les lectures d'Odometria al Conjunt de Navegació.
- **/tf: Proporciona les Transformacions al Conjunt de Navegació.**

11. Node move_base

Aquest és el node més important del Stack de Navegació. És on es produeix la major part de la "magia".

La funció principal del node **move_base** és moure un robot de la seva posició actual a una posició objectiu amb l'ajuda d'altres nodes de navegació. Aquest node enllaça el planificador global i el planificador local per a la planificació del camí, connectant amb el paquet de recuperació de gir si el robot queda atrapat en algun obstacle, i connectant el costmap global i el costmap local per obtenir el mapa d'obstacles de l'entorn.

A continuació, es mostra la llista de tots els paquets que estan enllaçats pel node move_base:

- global-planner
- local-planner
- rotate-recovery
- clear-costmap-recovery
- costmap-2D

A continuació, es mostren els altres paquets que interfereixen amb el node move_base:

- map-server
- AMCL
- gmapping

12. Resum

El Stack de Navegació és un conjunt de nodes i algorismes de ROS que treballen junts per moure un robot des d'una posició A fins a una posició B, evitant els obstacles que pugui trobar pel camí. Per fer això, el Conjunt de Navegació requereix diverses entrades de dades (de diferents tipus). A canvi, donarà com a sortida les velocitats de comandament necessàries per moure el robot de manera segura fins a la posició desitjada.